

FedRGL: Federated Riemannian Graph Learning in Mixed-Curvature Spaces with Ricci-Gated Convolution

Haizhou Du

Faculty of Artificial Intelligence
Shanghai University of Electric Power
Shanghai, China

Zijie Zhu

Faculty of Artificial Intelligence
Shanghai University of Electric Power
Shanghai, China

Haolin Wu

Faculty of Artificial Intelligence
Shanghai University of Electric Power
Shanghai, China

Zicheng Shi

Faculty of Artificial Intelligence
Shanghai University of Electric Power
Shanghai, China

Abstract

Federated Graph Learning (FGL) has emerged as an efficient paradigm to address the pronounced data heterogeneity common in real-world decentralized graph datasets, attracting significant interest from both academia and industry. Existing FGL methods often degrade the global model's performance due to a fundamental mismatch between the model's fixed-geometry embedding space and the diverse geometric structures of client data. To address this issue, we propose a novel framework for Federated Riemannian Graph Learning, namely FedRGL. FedRGL introduces a personalized mixed-curvature product space for each heterogeneous client, mapping local graph data into a tailored geometric space composed of Euclidean, hyperbolic, and spherical manifolds. Furthermore, it leverages a Ricci-Gated Graph Convolutional Network to dynamically adapt its message-passing mechanism to the local topology of each graph. Extensive experiments across diverse geometric heterogeneity settings demonstrate that FedRGL significantly outperforms state-of-the-art FGL methods in terms of model accuracy and generalization.

CCS Concepts

• **Computing methodologies** → **Distributed computing methodologies**; • **Computer systems organization** → *Distributed architectures*.

Keywords

Federated Graph Learning; Ricci Curvature; Riemannian Space

ACM Reference Format:

Haizhou Du, Haolin Wu, Zijie Zhu, and Zicheng Shi. 2026. FedRGL: Federated Riemannian Graph Learning in Mixed-Curvature Spaces with Ricci-Gated Convolution. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792142>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792142>

1 Introduction

In the realm of Graph Neural Networks (GNNs), most systems are designed for a unified, centralized graph [9]. However, real-world applications frequently involve individual users or institutions maintaining private graphs, isolated due to privacy concerns [37]. Federated Learning (FL) has emerged as a promising paradigm to address this, enabling collaborative model training across decentralized clients while preserving data privacy [12, 16]. Consequently, applying FL to GNNs, known as Federated Graph Learning (FGL), has garnered significant attention. It provides a solution by enabling clients to independently train local GNNs on their data, followed by a central server aggregating local updates to create a powerful global model [10, 18]. This decentralized approach is vital for a wide range of applications, from social networks and recommendation systems to biomedical data analysis [19, 34, 40].

The collaborative nature of FGL is particularly challenged by the non-independent and identically distributed (non-IID) data across clients. This heterogeneity is especially complex in graph data, as it manifests not only in features and labels but also in the underlying graph topology and geometric structure [36, 43], leaving several important challenges open for exploration.

The first challenge lies in **the limitation of single-curvature embedding spaces**. Most prior works, both in centralized and federated settings, model graph data in Euclidean space [12, 19]. While straightforward and computationally efficient, Euclidean geometry struggles to embed non-Euclidean structures like hierarchies without significant distortion [6, 21]. Recognizing this, recent studies have attempted to explore hyperbolic space for modeling graphs, as its exponential volume growth makes it a natural fit for tree-like and scale-free structures [2, 3, 29, 39]. However, these methods still rely on a single fixed-curvature space. Real-world graphs are often a mosaic of different local structures: tree-like branches, dense cyclical communities, and grid-like patterns can coexist within the same network [8, 28]. A single-curvature space is inherently limited in its capacity to faithfully represent all these diverse geometric patterns simultaneously [1, 21]. Therefore, we argue that a more expressive mixed-curvature product space is necessary to comprehensively learn the rich and varied geometric information present in graph data [8, 27].

The second challenge is **the lack of fine-grained topological awareness** in the learning process. While some FGL methods attempt to handle structural differences by extracting common

features or addressing missing neighbors [35, 43], they often overlook the rich local geometric information that distinguishes one client’s graph from another. In parallel, recent advances in centralized graph representation learning have shown that incorporating richer structural descriptors, such as **discrete Ricci curvature**, can significantly enhance a model’s understanding of graph topology [26, 27, 41]. An edge’s Ricci curvature, by measuring the “connectedness” of its local neighborhood, provides a powerful descriptor of local structure that goes beyond simple metrics like node degree [22, 31]. However, the application of such advanced geometric concepts within the federated learning paradigm remains unexplored. Therefore, we propose applying discrete Ricci curvature to federated graph learning to effectively capture the local topological structures of heterogeneous client graphs.

Motivated by the aforementioned challenges, we propose FedRGL, a **novel Federated Riemannian Graph Learning** framework. To address the first challenge, we design a personalized heterogeneous curvature space for each client, constructed as a product of *Hyperbolic, Spherical, and Euclidean manifolds*. The curvature of each component manifold is learned and trained locally, allowing the embedding space to adapt to the client’s specific data geometry. To tackle the second challenge, we introduce a Ricci Curvature-Aware Graph Convolutional Network (Ricci-Gated GCN). This network utilizes discrete Ricci curvature to create a dynamic gating mechanism that modulates the message-passing process, enabling the model to learn representations that are sensitive to the local topological context of each edge.

Our **main contributions** can be summarized as follows:

- We are the first to introduce a personalized, mixed-curvature product space into the federated graph learning framework, namely FedRGL. This FedRGL can faithfully capture the diverse geometric structures of each client’s heterogeneous graph data by learning client-specific curvatures.
- We propose a novel Ricci-Gated GCN architecture that integrates discrete Ricci curvature into the message-passing mechanism. This enables a more fine-grained and topology-aware learning of graph structure within the FL paradigm.
- Extensive experiments on various graph datasets demonstrate that FedRGL significantly outperforms state-of-the-art baselines, especially under settings with high geometric heterogeneity.

2 Related Work

2.1 Federated Graph Learning

FGL has emerged as a crucial field to address privacy concerns and data silos in real-world graph applications, where data is naturally decentralized across multiple clients [10, 18]. FGL frameworks can generally be categorized into two main types: inter-graph and intra-graph learning. In inter-graph FGL, clients, each holding a distinct set of graphs, collaborate to train a GNN model, often tackling challenges arising from non-IID graph distributions [2, 39, 41]. Intra-graph FGL, deals with a single logical graph whose nodes and edges are partitioned across clients. In this setting, a primary challenge is handling the missing cross-client edges. To address this, some methods propose techniques to explicitly reconstruct or approximate the influence of these missing neighbors, for example, by sharing

neighbor embeddings or generating synthetic neighbors [44]. To handle the broader issue of data heterogeneity, other approaches have explored personalized federated learning (PFL) for graphs. These include methods like creating personalized masks [15], separating models into global and local components [43], or clustering clients based on model similarity [7]. However, a significant limitation of nearly all existing FGL works is their reliance on Euclidean space for graph embedding. This assumption fails to adequately capture the complex geometric structures inherent in many real-world graphs, especially under the pronounced heterogeneity of the federated setting. Our work moves beyond this limitation by introducing personalized, mixed-curvature Riemannian spaces to better model the diverse geometries of client data.

2.2 Riemannian Graph Learning

Riemannian Graph Learning has recently gained significant attention as a powerful alternative to traditional Euclidean methods, offering a more expressive geometric inductive bias for graph-structured data. A substantial body of work has focused on leveraging hyperbolic space for its natural alignment with tree-like and scale-free structures commonly found in real-world networks [30, 42, 46]. Concurrently, spherical space has been explored for its suitability in modeling cyclical patterns and community structures [1, 45]. Recognizing that real graphs often contain a mix of these patterns, recent research has shifted towards mixed-curvature spaces, typically constructed as a product of multiple constant-curvature manifolds. These models, such as those proposed in [8, 28, 38], can simultaneously capture diverse geometric properties within a single graph. Furthermore, some works have started to explore the use of discrete curvatures, like Ricci curvature, to guide the GNN’s message-passing process, thereby making the model more aware of the graph’s local topology [26, 47]. However, the vast majority of existing Riemannian graph learning methods are designed for a centralized learning paradigm, where the entire graph is accessible for training. The exploration of these advanced geometric techniques within a distributed, privacy-preserving federated learning setting is still in its infancy. Therefore, we apply these powerful Riemannian geometry concepts to the FGL domain to better enhance the performance of personalized client models.

3 Preliminaries

3.1 Riemannian Geometry

Riemannian Manifold. A Riemannian manifold $(\mathcal{M}, g)$ is a smooth manifold $\mathcal{M}$ equipped with a Riemannian metric g . For each point $x \in \mathcal{M}$, the metric g_x defines an inner product on the tangent space $\mathcal{T}_x \mathcal{M}$, which is a Euclidean vector space that serves as the local linear approximation of the manifold at x . The collection of all tangent spaces over the manifold is known as the tangent bundle $\mathcal{T}\mathcal{M}$. Two essential operations connect the manifold and its tangent spaces:

- The exponential map $\exp_x : \mathcal{T}_x \mathcal{M} \rightarrow \mathcal{M}$ projects a vector $v \in \mathcal{T}_x \mathcal{M}$ from the tangent space onto the manifold.
- The logarithmic map $\log_x : \mathcal{M} \rightarrow \mathcal{T}_x \mathcal{M}$ is the inverse of the exponential map, projecting a point $y \in \mathcal{M}$ back to the tangent space at x .

These maps allow us to leverage the well-defined vector space structure of the tangent space for operations like linear transformations and aggregations, before mapping the results back onto the manifold.

Curvature and Curvature Spaces. For each point x in a manifold, it is associated with a curvature K_x which describes how the geometry of space around x deviates from being flat. When the curvature K_x is constant for all points across the manifold, it induces a **Constant Curvature Space (CCS)**. There are three canonical types of CCS, distinguished by the sign of their curvature K :

- **Hyperbolic Space ($\mathcal{H}^d$):** Characterized by a constant negative curvature ($K < 0$). Its volume grows exponentially with the radius, making it well-suited for embedding tree-like and hierarchical structures with low distortion.
- **Hyperspherical Space ($\mathcal{S}^d$):** Characterized by a constant positive curvature ($K > 0$). Its geometry is ideal for modeling cyclical or community-based structures.
- **Euclidean Space ($\mathbb{R}^d$):** This is the special case with zero curvature ($K = 0$) and serves as the default space for most standard machine learning models.

3.2 Graph Curvature

Discrete graph curvature adapts concepts from Riemannian geometry to graphs, providing a powerful way to quantify the local topology of a network. It offers a richer structural description than simple node-based metrics like degree [25]. In this work, we focus on Forman-Ricci curvature for its computational efficiency and intuitive combinatorial definition [33].

Forman-Ricci Curvature. The Forman-Ricci curvature, introduced by Robin Forman [5], is a versatile and computationally efficient definition of discrete curvature. For an edge $e = (u, v)$ in a simple, unweighted graph, a basic and intuitive form of the Forman-Ricci curvature $F(e)$ or F_{uv} is defined by a combinatorial formula:

$$F_{uv} = 4 - (\deg(u) + \deg(v)), \quad (1)$$

where $\deg(u)$ and $\deg(v)$ are the degrees of the nodes u and v , respectively. Equation(1) captures a core intuition: edges connecting high-degree nodes are likely to be part of dense regions and thus have lower (more negative) curvature, indicating a “stretching” of the local geometry.

To provide a more precise feature, especially in graphs with higher-order structures like triangles, the definition is often augmented. A common and powerful extension for an unweighted graph is:

$$F_{uv} = 4 - (\deg(u) + \deg(v)) + 3 \times |\mathcal{T}_{uv}|, \quad (2)$$

where $|\mathcal{T}_{uv}|$ is the number of triangles that contain the edge (u, v) . This version directly rewards edges that are part of closed three-node loops by assigning them higher (more positive) curvature.

4 Methodology

In this section, we present the methodology of Federated Riemannian Graph Learning (FedRGL) framework. Section 4.1 defines the federated graph learning problem. Section 4.2 provides the overview of FedRGL architecture. Section 4.3 and Section 4.4 details the two main components of FedRGL.

4.1 Problem Statement

Given a set of c clients, denoted by $C = \{1, 2, \dots, C\}$, where each client c possesses a private local graph dataset $\mathcal{G}_c = (x_i^c, y_i^c)_{i=1}^{N_c}$ and distribution $p_c(G, Y)$. The goal of FGL is to collaboratively train a global graph neural network model, parameterized by θ , by aggregating insights from all clients without centralizing their data. However, a significant challenge in this setting is data heterogeneity, where for any client pair $i \neq j$, it is often the case that $p_i(G, Y) \neq p_j(G, Y)$. This non-IID problem is particularly severe for graph data due to its higher complexity, as heterogeneity can manifest not only in features and labels but also in the underlying graph topology and geometric structure. To address this, we adopt a personalized federated learning approach, where the model parameters are decoupled into shared parameters θ_s and client-specific personalized parameters. In FedRGL framework, we specifically designate the parameters defining the geometric space as personalized. Our objective is to jointly optimize the shared model parameters θ_s and the set of personalized curvature parameters $\{\mathcal{K}_c\}_{c=1}^C$ to minimize a global loss function:

$$\min_{\{\mathcal{K}_c\}_{c=1}^C, \theta_s} \sum_{c=1}^C w_c \mathcal{L}_c(f(G_c; \theta_s, \mathcal{K}_c), Y_c), \quad (3)$$

where $f(\cdot)$ is the local GNN model, $\mathcal{L}_c(\cdot)$ is the local loss function on client c , and w_c is the weight of client c in the global aggregation.

Within this FGL context, we identify two critical and under-explored challenges. The first is the geometric heterogeneity of client graphs. Existing methods typically assume a single, fixed-curvature embedding space (e.g., Euclidean), which fails to capture the diverse geometries of client data, such as tree-like hierarchies or dense cyclical communities. The second challenge is the lack of fine-grained topological awareness in the learning process. Standard GNNs often overlook the rich local structural information encoded in the graph’s topology, treating all connections in the neighborhood uniformly. While recent works in centralized learning have explored discrete Ricci curvature to capture this information, its application in the FGL paradigm remains unaddressed. Therefore, we leverage Ricci curvature and contrastive learning to better learn this information.

4.2 The Overview of FedRGL

To address the above challenges, as illustrated in Figure 1, FedRGL tackles the dual challenges of geometric heterogeneity and local topological complexity. **Graph Embedding in Heterogeneous Curvature Spaces** (Section 4.3) addresses geometric heterogeneity by enabling each client to learn a personalized mixed-curvature product space by locally training its own curvature parameters. **Geometry-aware Graph Convolutional Layer** (Section 4.4) captures local topological complexity, which incorporates a Ricci-gated Convolution to make message-passing sensitive to local structure, followed by Geometric Contrasting to ensure semantic consistency across different geometric views.

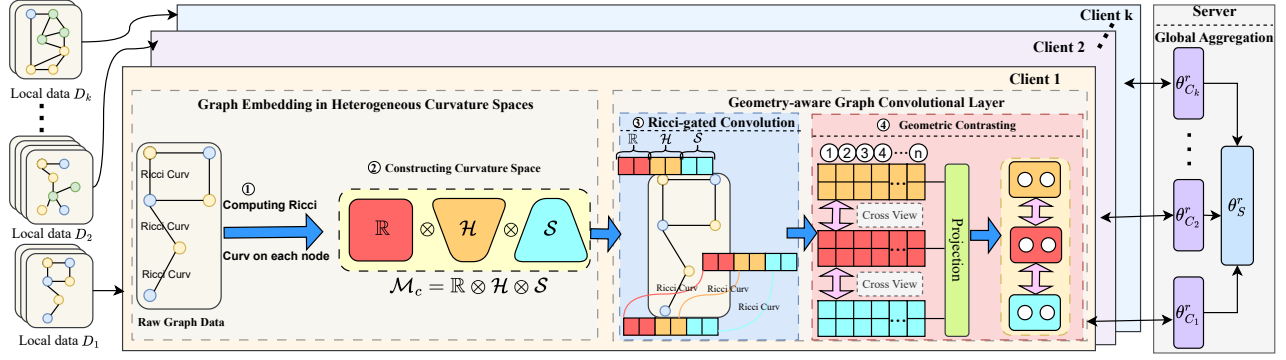


Figure 1: The Architecture of FedRGL

4.3 Graph Embedding in Heterogeneous Curvature Spaces

To address the challenge of geometric heterogeneity, we move beyond single-geometry embeddings and propose a personalized embedding space for each client. This approach is motivated by the observation that real-world graphs exhibit complex and mixed structural patterns; they are often tree-like in some regions and cyclical in others. While hyperbolic space excels at capturing hierarchical structures and spherical space is ideal for cyclical patterns, neither can fully represent a graph with mixed geometries. Therefore, to enable a more faithful and comprehensive representation of each client's unique graph data, we introduce a **mixed-curvature product manifold** for graph embedding.

Personalized Product Manifold Construction. For each client c , we construct a personalized product manifold $\mathcal{M}_c$ as the Cartesian product of a Euclidean space factor, a hyperbolic space factor, and a hyperspherical space factor. The product manifold is formally defined as:

$$\mathcal{M}_c = \mathbb{R}^{d_e} \otimes \mathcal{H}_{K_c^h}^{d_h} \otimes \mathcal{S}_{K_c^s}^{d_s}, \quad (4)$$

where $\otimes$ denotes the Cartesian product of manifolds. $\mathbb{R}^{d_e}$ is a d_e -dimensional Euclidean space, $\mathcal{H}_{K_c^h}^{d_h}$ is a d_h -dimensional hyperbolic space, and $\mathcal{S}_{K_c^s}^{d_s}$ is a d_s -dimensional hyperspherical space. The parameters that define this space are:

- Dimensions $\{d_e, d_h, d_s\}$: These are architectural hyperparameters, shared across all clients.
- Curvature Parameters $\mathcal{K}_c = \{K_c^h, K_c^s\}$: These are the key to personalization. $K_c^h > 0$ defines the negative curvature $-1/K_c^h$ of the hyperbolic component, and $K_c^s > 0$ defines the positive curvature $1/K_c^s$ of the spherical component. These curvature parameters are client-specific and personalized. They are learned and stored locally on each client's device and do not participate in server-side aggregation. This allows the geometry of the embedding space to adaptively reflect the local structural information of client c 's graph data.

Consequently, a point (node representation) $\mathbf{h}_u \in \mathcal{M}_c$ for a node u is represented by the concatenation of its components in each

factor manifold:

$$\mathbf{h}_u = [\mathbf{h}_u^e || \mathbf{h}_u^h || \mathbf{h}_u^s], \quad (5)$$

where $\mathbf{h}_u^e \in \mathbb{R}^{d_e}$, $\mathbf{h}_u^h \in \mathcal{H}_{K_c^h}^{d_h}$, $\mathbf{h}_u^s \in \mathcal{S}_{K_c^s}^{d_s}$ and $||$ denotes vector concatenation.

Geometric-enhanced Feature Initialization. We initialize the node representations by embedding the raw Euclidean features $\mathbf{x}_u \in \mathbb{R}^d$ into the personalized product manifold $\mathcal{M}_c$. This process creates an initial “geometry-enhanced” representation for each node. The mapping is performed by first projecting the raw features into subspaces corresponding to each manifold factor and then using the respective exponential map $\exp_o(\cdot)$ at the origin for the Riemannian components. For the Euclidean component, the mapping is a standard linear transformation:

$$\mathbf{h}_u^{e,(0)} = \mathbf{W}_e \mathbf{x}_u. \quad (6)$$

For Riemannian components, the mapping is formally written as:

$$\mathbf{h}_u^{h,(0)} = \exp_{o, K_c^h}(\mathbf{W}_h \mathbf{x}_u). \quad (7)$$

$$\mathbf{h}_u^{s,(0)} = \exp_{o, K_c^s}(\mathbf{W}_s \mathbf{x}_u). \quad (8)$$

The initial mixed-curvature representation is then formed by the concatenation of these components: $\mathbf{h}_u^{(0)} = [\mathbf{h}_u^{e,(0)} || \mathbf{h}_u^{h,(0)} || \mathbf{h}_u^{s,(0)}]$.

In this initialization step, the projection matrices $\mathbf{W}_e, \mathbf{W}_h, \mathbf{W}_s$ are part of the shared parameters θ_s and are learned collaboratively through federated aggregation. In contrast, the curvature parameters K_c^h and K_c^s used in the exponential maps are the personalized parameters $\mathcal{K}_c$. During local training on client c , these personalized curvature parameters are updated independently using a Riemannian optimizer (e.g., Riemannian Adam) and are never sent to the server. This design ensures that the geometric shape of each client's embedding space is tailored to its local data, while the feature projection mechanism benefits from all clients' knowledge.

4.4 Geometry-aware Graph Convolutional Network

To enable the GNN to perceive and adapt to the local topology of the graph, we introduce the Ricci-Gated GCN. This network enhances the standard message-passing mechanism by incorporating discrete Ricci curvature, a powerful measure of local graph structure.

Discrete Ricci Curvature: A Geometric Intuition. Discrete Ricci curvature provides a geometric characterization of edges within a graph. There are several definitions, with Ollivier-Ricci [17] and Forman-Ricci [11] being the most prominent. At its core, an edge’s Ricci curvature describes the overlap between the neighborhoods of its two endpoints, thus signifying the density and connectivity of the local structure [20].

- **Positive Ricci Curvature:** An edge with high positive curvature typically lies within a dense cluster or clique. This indicates that its endpoints’ neighborhoods are heavily overlapped, facilitating easy information flow.
- **Negative Ricci Curvature:** An edge with negative curvature often acts as a “bridge” connecting two sparser communities. Its endpoints’ neighborhoods are less overlapped, suggesting that information transfer across this edge should be more cautious [23].

In FedRGL framework, we leverage this geometric intuition to improve the message-passing mechanism. We pre-compute a discrete Ricci curvature value $Ric_c(u, v)$ for each edge (u, v) on the client’s local graph G_c .

Ricci-Gated Message Passing in the Product Manifold. The message-passing and aggregation process of our GNN is performed directly within the personalized product manifold $\mathcal{M}_c$. We update the node representations layer by layer from the geometry-enhanced initial embeddings. The operations are performed independently for each component of the product space (Euclidean, Hyperbolic, and Spherical), and all operations are designed to be manifold-preserving.

The update from layer l to $l + 1$ for a target node u is conducted as follows:

(1) **Component-wise Linear Transformation.** For each component $(k \in \{e, h, s\})$, we first transform the node representation $\mathbf{h}_v^{k,(l)}$ using a manifold-specific linear operation:

$$\mathbf{m}_v^{k,(l+1)} = \mathbf{W}^{k,(l)} \otimes_{K_c^k} \mathbf{h}_v^{k,(l)}, \quad (9)$$

where $\mathbf{W}^{k,(l)} \in \theta_s$ is a shared weight matrix, and $\otimes_{K_c^k}$ denotes the appropriate linear transformation for that geometry (e.g., standard matrix multiplication for Euclidean, Möbius matrix-vector multiplication for hyperbolic).

(2) **Ricci-Gated Aggregation Weight Generation.** We introduce a Ricci gate $g_{uv}^{(l+1)}$ to generate the message aggregation weight. This weight is determined directly by the edge’s topological role, as measured by Ricci curvature, and modulated through a data-driven learning process:

$$g_{uv}^{(l+1)} = \text{MLP}^{(l)}(Ric_c(u, v)), \quad (10)$$

where $\text{MLP}^{(l)}$ is a lightweight multi-layer perceptron whose parameters are part of θ_s . This MLP learns a non-linear mapping from the Ricci curvature value to an aggregation importance score. We then normalize these gate values across the neighborhood to obtain the final aggregation weights $\alpha_{uv}^{(l+1)}$:

$$\alpha_{uv}^{(l+1)} = \text{Softmax}_{j \in \mathcal{N}(u) \cup \{u\}} (g_{uj}^{(l+1)}), \quad (11)$$

note that the weight α_{uv} is driven entirely by the local geometric structure, not node features.

(3) **Component-wise Gated Aggregation.** The aggregation process is performed via a **generalized addition**, which is native to the manifold:

$$\mathbf{a}_u^{k,(l+1)} = \bigoplus_{v \in \mathcal{N}(u) \cup \{u\}}^{K_c^k} \alpha_{uv}^{(l+1)} \otimes_{K_c^k} \mathbf{m}_v^{k,(l+1)}. \quad (12)$$

The Equation(12) for the aggregation and scalar multiplication $\otimes_{K_c^k}$ is detailed in Appendix B, drawing from established formalisms. This process yields an aggregated representation $\mathbf{a}_u^{k,(l+1)}$ for each geometric component.

(4) **Component-wise Update.** Finally, each component is updated with a residual connection and a manifold-preserving activation function $\sigma^\otimes$:

$$\mathbf{h}_u^{k,(l+1)} = \left(\sigma^\otimes(\mathbf{a}_u^{k,(l+1)}) \right) \oplus_{K_c^k} \mathbf{h}_u^{k,(l)}. \quad (13)$$

After these steps, the updated component representations are ready for the next layer.

Feature Refinement via Geometric Contrastive Learning. After the Ricci-gated message passing at each layer, the updated representations from the different geometric views capture both local topology and diverse structural information. To ensure these views are semantically aligned, we perform geometric contrastive learning. First, we project the updated representations from the different Riemannian views (hyperbolic and spherical) into a **common tangent space** at the origin using the logarithmic map $\log_o(\cdot)$:

$$\mathbf{z}_u^{h,(l+1)} = \log_{o,K_c^h}(\mathbf{h}_u^{h,(l+1)}), \quad \mathbf{z}_u^{s,(l+1)} = \log_{o,K_c^s}(\mathbf{h}_u^{s,(l+1)}). \quad (14)$$

The Euclidean component, $\mathbf{z}_u^{e,(l+1)} = \mathbf{h}_u^{e,(l+1)}$, already resides in a compatible vector space. We then contrast each Riemannian view with the Euclidean view using the InfoNCE loss. The total contrastive loss for layer $l + 1$ is:

$$\mathcal{L}_{GC}^{(l+1)} = \mathcal{L}_{E-H}^{(l+1)} + \mathcal{L}_{E-S}^{(l+1)}, \quad (15)$$

where $\mathcal{L}_{E-H}$ and $\mathcal{L}_{E-S}$ are the InfoNCE losses between the Euclidean-hyperbolic and Euclidean-spherical views, respectively. For example, $\mathcal{L}_{E-H}$ is defined as:

$$\mathcal{L}_{E-H}^{(l+1)} = - \sum_{i=1}^N \log \frac{\exp(\text{sim}(\mathbf{z}_i^{e,(l+1)}, \mathbf{z}_i^{h,(l+1)})/\tau)}{\sum_{j=1}^N \exp(\text{sim}(\mathbf{z}_i^{e,(l+1)}, \mathbf{z}_j^{h,(l+1)})/\tau)}. \quad (16)$$

This contrastive loss, summed over all layers, is added to the client’s local objective, encouraging the Euclidean representation to absorb the rich geometric information learned in the other manifolds.

5 Experiments

5.1 Datasets

We evaluate FedRGL on four graph datasets spanning chemical and academic domains, including citation networks Cora [24] and CiteSeer [24] for document categorization, and molecular graphs NCI1 [32] and Mutagenicity [13] for chemical property prediction, to assess its performance across diverse structural characteristics and learning tasks.

Algorithm 1 Federated Riemannian Graph Learning (FedRGL)

```

1: Input: Initial personalized curvature parameters  $\{\mathcal{K}_{c_i}^{(0)}\}_{i=1}^C$ ,
   initial shared parameters  $\theta_s^{(0)}$ , client datasets  $\{\mathcal{D}_{c_i}\}_{i=1}^C$ , learn-
   ing rate  $\eta$ .
2: Output: Trained shared parameters  $\theta_s$ .
3: Server executes:
4: Initialize shared parameters  $\theta_s \leftarrow \theta_s^{(0)}$ .
5: for each communication round  $r = 0, 1, \dots, R - 1$  do
6:   for each client  $c_i \in C$  in parallel do
7:      $\theta_{c_i}^{(r+1)} \leftarrow \text{ClientUpdate}(c_i, \theta_s^{(r)}, \mathcal{K}_{c_i}^{(r)})$ .
8:   end for
9:   Aggregate shared parameters:  $\theta_s^{(r+1)} \leftarrow \sum_{i=1}^C w_{c_i} \theta_{c_i}^{(r+1)}$ .
10: end for
11:
12: ClientUpdate( $c_i, \theta_s, \mathcal{K}_{c_i}$ ):
13:  $\theta_{c_i}^{(r)} \leftarrow \theta_s^{(r)}$ .
14: if  $r == 0$  then
15:   Initialize personalized curvatures  $\mathcal{K}_{c_i} \leftarrow \mathcal{K}_{c_i}^{(0)}$ .
16: end if
17: Pre-compute discrete Ricci curvature  $\text{Ric}_{c_i}(u, v)$  for graph  $G_{c_i}$ .
18: for local epochs  $e = 1, \dots, E$  do
19:   // 1. Geometric-Enhanced Feature Initialization (Eq. 4-8)
20:    $\mathbf{h}_u^{(0)} \xleftarrow{\text{InitializeFeatures}} (\mathbf{x}_u; \theta_{c_i}, \mathcal{K}_{c_i})$  for all  $u \in G_{c_i}$ .
21:    $\mathcal{L}_{GC} \leftarrow 0$ .
22:   for each GCN layer  $l = 0, \dots, L - 1$  do
23:     // 2. Ricci-Gated Message Passing (Eq. 9-14)
24:      $\{\mathbf{h}_u^{k, (l+1)}\}_{k \in \{e, h, s\}} \xleftarrow{\text{RicciGatedLayer}} (\{\mathbf{h}_u^{k, (l)}\}_{k \in \{e, h, s\}})$ .
25:     // 3. Geometric Contrastive Learning (Eq. 15-16)
26:      $\mathcal{L}_{GC} \leftarrow \mathcal{L}_{GC} + \mathcal{L}_{E-H}^{(l+1)} + \mathcal{L}_{E-S}^{(l+1)}$ .
27:   end for
28:   Compute total local loss:  $\mathcal{L}_{c_i} = \mathcal{L}_{task} + \lambda_{gc} \mathcal{L}_{GC}$ .
29:   Update local parameters via gradient descent:
30:    $(\theta_{c_i}^{(r+1)}, \mathcal{K}_{c_i}^{(r+1)}) \leftarrow (\theta_{c_i}^{(r)}, \mathcal{K}_{c_i}^{(r)}) - \eta \nabla_{\theta_{c_i}^{(r)}, \mathcal{K}_{c_i}^{(r)}} \mathcal{L}_{c_i}$ .
31: end for
32: return  $\theta_{c_i}^{(r+1)}$ .

```

Table 1: Statistics of the datasets.

Datasets	Number of graphs	Features	Classes
Cora	2708	1433	7
CiteSeer	3312	3703	6
NCI1	4100	37	2
Mutagenicity	4337	14	2

5.2 Baselines

To evaluate the effectiveness of FedRGL, we select six FL methods for comparison on graph neural networks.

- **FedAvg** [19] is a classical FL framework. In our study, we employ GCN as the model within this framework to better capture graph structural information.

- **FedProx** [16] enhances FedAvg by adding a proximal term to the local objective function. We implement it with GCN as the local model to improve the training graph data.
- **FedSage+** [43]: FGL framework using GraphSage with graph mending to enhance subgraph connectivity and classification performance.
- **GCFL** [38]: Dynamically clusters clients via GNN gradients to address non-IID graph structures and features for federated training.
- **FedHGCN** [4]: Employs hierarchical feature extraction and node selection to improve classification accuracy and attack robustness in federated graph learning.
- **FedLoG** [14]: Condenses knowledge from head nodes into global synthetic data for adaptive generalization in subgraph FL.

5.3 Experimental Setup

Experiments were conducted on a workstation with an Intel Xeon Gold 6430 CPU, NVIDIA RTX 4090 GPU (24GB VRAM), and 120GB RAM, using Python 3.10 and PyTorch 2.5.1.

For the graph classification task, we employ a batch size of 32 and utilize the RiemannianAdam optimizer with a learning rate of 0.001, a dropout rate of 0.001, and a weight decay of $5e-4$. For the Riemannian geometric components, we initialize the hyperbolic curvature parameter to -1.0 and the spherical curvature to 1.0, with these curvature parameters being learnable and thus subject to change during training. We also configure a geometric learning rate of 0.01 and a curvature gradient clipping threshold of 0.1. The Ricci-gated graph convolution employs a temperature coefficient of 0.1 for contrastive learning, while the FL framework operates for 200 communication rounds across all participating clients.

To evaluate FedRGL under IID and non-IID settings, we partition graph datasets across clients using a Dirichlet distribution governed by parameter α . Smaller α values increase data heterogeneity, while larger α values promote uniformity. We test four scenarios: $\alpha = 0.1$, $\alpha = 0.5$, $\alpha = 1$ (non-IID to different degrees) and $\alpha = 10$ (IID).

5.4 Performance of accuracy experiment

In this experimental setup, we evaluate FedRGL across ten and twenty clients under IID data with Dirichlet parameter $\alpha = 10$. The datasets are evenly partitioned among participating clients to create homogeneous learning environments.

As shown in Table 2, FedRGL consistently outperforms all baseline methods across the four benchmark datasets, demonstrating its superior performance against state-of-the-art methods. The most remarkable improvement is observed on the CiteSeer dataset with 10 clients, where FedRGL achieves 73.36% accuracy – an impressive 10.3% increase over the strongest baseline FedHGCN. Similarly significant gains are seen on NCI1 with 10 clients (76.75%, +4.5% over FedHGCN) and 20 clients (74.87%, +5.6% over FedHGCN), highlighting FedRGL’s exceptional capability in molecular graph learning. Across all scenarios, FedRGL maintains consistent performance advantages ranging from 0.9% to 10.3%, establishing itself as the top-performing method in every configuration.

Table 2: Accuracy comparison of federated learning methods across multiple datasets and client numbers in IID scenarios

# clients	Cora		CiteSeer		NCI1		Mutagenicity	
	10	20	10	20	10	20	10	20
FedAvg	79.98%	85.25%	63.80%	67.41%	66.39%	68.06%	76.29%	73.21%
FedProx	81.31%	83.01%	64.03%	67.21%	67.20%	67.39%	76.18%	72.85%
FedSage+	81.60%	82.41%	62.13%	66.69%	68.60%	68.42%	77.35%	72.58%
GCFL	<u>81.38%</u>	<u>84.47%</u>	62.55%	66.54%	66.34%	66.26%	76.52%	73.88%
FedHGCN	81.06%	<u>85.76%</u>	<u>66.82%</u>	66.28%	<u>73.45%</u>	<u>70.89%</u>	<u>78.88%</u>	<u>76.33%</u>
FedLoG	80.26%	83.11%	63.33%	67.50%	66.30%	66.38%	75.40%	73.92%
FedRGL	83.80(↑2.9%)	86.88(↑1.3%)	73.36(↑10.3%)	69.69(↑3.2%)	76.75(↑4.5%)	74.87(↑5.6%)	79.57(↑0.9%)	77.68(↑1.7%)

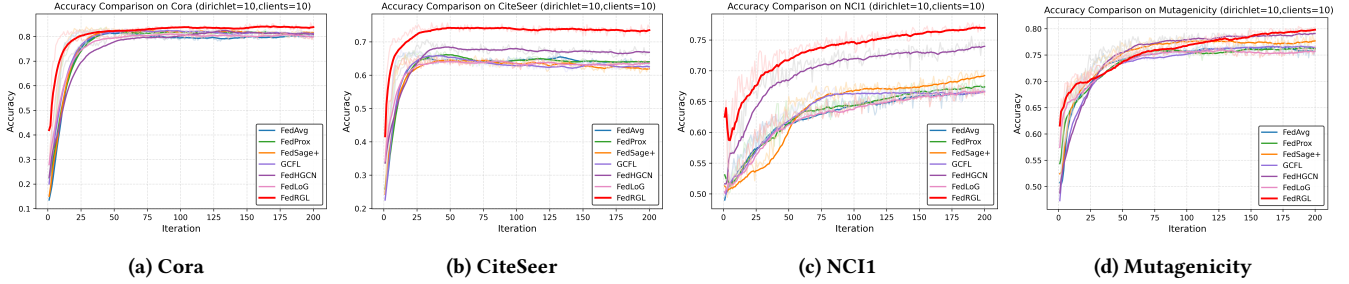
**Figure 2: Accuracy comparison of FedRGL and baseline methods across multiple graph datasets under IID scenarios ($\alpha = 10$)**

Figure 2 presents the accuracy comparison across multiple datasets under IID settings, providing a comprehensive visualization of FedRGL’s superior performance. On NCI1 (10 clients), FedRGL not only achieves the highest final accuracy but also exhibits stable convergence behavior throughout the training process. Similarly, on CiteSeer (10 clients), FedRGL shows a substantial performance gap over all competing methods. The remaining subfigures further confirm FedRGL’s consistent superiority on Cora and Mutagenicity datasets, completing the picture of comprehensive dominance across diverse graph learning tasks.

This comprehensive superiority across different network structures and client numbers makes FedRGL particularly suitable for federated graph learning applications where both reliability and accuracy are paramount. The consistent performance advantage visible across all experimental settings further confirms the effectiveness and generalization capability of our approach.

5.5 Evaluation under non-IID data distributions

To comprehensively assess the stability of FedRGL in realistic federated learning scenarios, we evaluate FedRGL under varying levels of data heterogeneity using citation (Cora) and biochemical (NCI1) graphs, with 10 to 20 clients and Dirichlet (α) from 0.1 (high heterogeneity) to 1.0 (moderate heterogeneity).

On the Cora dataset, as detailed in Table 3, FedRGL demonstrates remarkable performance consistency across all experimental configurations. The largest improvement is observed under the challenging setting of high data heterogeneity ($\alpha = 0.1$) with 20 clients, where FedRGL achieves 87.86% accuracy - a 6.4% increase over FedHGCN. Under moderate heterogeneity ($\alpha = 0.5$) with 20 clients, it reaches 91.44%, outperforming FedHGCN by 5.6%. Notably, FedRGL

consistently surpasses the best baselines by 2.3% to 6.4% across all settings, with strong results in challenging non-IID scenarios.

The NCI1 results, visualized in Figure 3, validate FedRGL’s superiority in handling the graph data under non-IID conditions. Across all three heterogeneity levels (10 clients), FedRGL consistently attains the highest accuracy. In the most heterogeneous scenario ($\alpha = 0.5$), it achieves 83.7%, a 12.1% improvement over FedHGCN. The performance gap remains evident across moderate ($\alpha = 0.5$) and IID-like ($\alpha = 1.0$) settings, maintaining clear advantages of 11.4% and 17.3% over FedSAGE+ and GCFL, respectively.

These findings demonstrate FedRGL’s stable and superior performance regardless of data distribution. Its robustness to heterogeneity and consistent results across graph types and client numbers make it an effective solution for real-world federated graph learning with non-IID data.

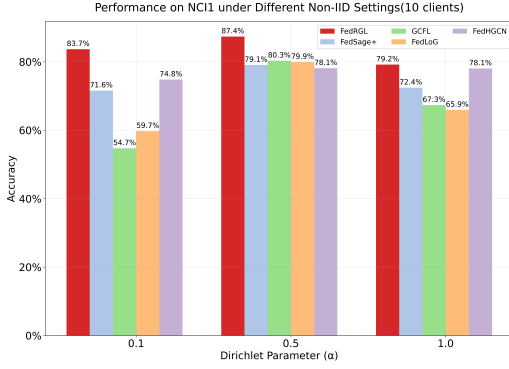
5.6 Ablation study on model components

To analyze the contribution of each component in FedRGL, we conduct comprehensive ablation studies under the 10-client setting with Dirichlet allocation ($\alpha = 10$). The convergence performance on NCI1 is illustrated in Figure 4, and quantitative results across three datasets are summarized in Table 4. These results collectively demonstrate the necessity and effectiveness of our core designs.

The Foundation of Mixed-Curvature Embedding. The variant **FedRGL w/o RC&CL** (FedRGL-mix), which retains only the mixed-curvature embedding while removing both the Ricci curvature gating and contrastive learning, establishes a solid performance baseline. It achieves 69.50% on NCI1, 75.37% on Mutagenicity, and 81.91% on Cora, outperforming the FedAvg across all datasets. This confirms that the product manifold serves as a powerful geometric

Table 3: Cora performance under varying non-IID settings

# clients	$\alpha = 0.1$		$\alpha = 0.5$		$\alpha = 1$	
	10	20	10	20	10	20
FedAvg	66.18%	62.54%	79.56%	71.09%	76.28%	79.26%
FedProx	58.75%	58.42%	81.33%	71.44%	75.54%	78.07%
FedSage+	68.56%	61.78%	79.24%	69.17%	75.44%	80.66%
GCFL	68.45%	62.29%	80.09%	71.14%	78.18%	84.34%
FedHGCN	82.09%	82.57%	81.07%	86.60%	83.47%	85.17%
FedLoG	61.49%	63.31%	81.56%	70.64%	80.86%	82.55%
FedRGL	84.36% ($\uparrow 2.7\%$)	87.86% ($\uparrow 6.4\%$)	84.51% ($\uparrow 3.6\%$)	91.44% ($\uparrow 5.6\%$)	85.78% ($\uparrow 2.7\%$)	87.14% ($\uparrow 2.3\%$)

**Figure 3: NCI1 performance under different non-IID settings**

foundation, enhancing the model’s representational capacity for heterogeneous graph structures. However, its performance gap relative to the complete FedRGL underscores that geometric embedding alone is insufficient without specialized mechanisms for structural filtering and representation regularization.

The Structural Enhancement via Ricci Curvature Gating. The variant **FedRGL w/o CL** (FedRGL-curv), which incorporates Ricci curvature-gated message passing but omits contrastive learning, demonstrates remarkable improvements, particularly on the NCI1 dataset, where it reaches 74.61% - a substantial 5.11% gain over FedRGL-mix. This significant enhancement highlights the critical role of curvature-aware gating in leveraging geometric properties to weight information flow. This mechanism provides exceptional stability and performance gains, especially evident in biomedical graph datasets like NCI1.

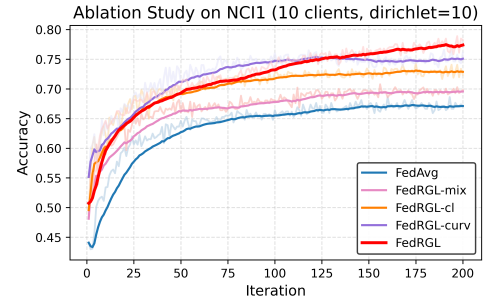
The Regularization Power of Contrastive Learning. The variant **FedRGL w/o RC** (FedRGL-cl), employing mixed-curvature embedding with contrastive learning but without Ricci curvature gating, achieves robust performance across all benchmarks (72.88% on NCI1, 76.51% on Mutagenicity, and 82.59% on Cora). Notably, it outperforms FedRGL-curv on Mutagenicity and Cora, demonstrating that contrastive loss effectively regularizes representations by enforcing intra-class compactness and inter-class separation.

Synergistic Integration for Optimal Performance. The complete **FedRGL** model consistently achieves the highest accuracy across all datasets, attaining 77.02% on NCI1, 79.77% on Mutagenicity, and 85.33% on Cora. The exceptional performance advantage is

particularly pronounced on NCI1, where FedRGL outperforms the second-best variant by 2.41 percentage points. As shown in Figure 4, FedRGL not only converges to superior final accuracy but also maintains the most stable and efficient learning trajectory throughout training. This outstanding performance stems from the synergistic integration of all three components: the mixed-curvature embedding, the Ricci curvature gating, and the contrastive learning.

Table 4: Ablation Study on FedRGL (10 clients, $\alpha = 10$)

	Cora	Mutagenicity	NCI1
FedAvg	79.48%	67.12%	72.70%
FedRGL-mix	81.91%	69.50%	75.37%
FedRGL-curv	82.55%	74.61%	75.86%
FedRGL-cl	82.59%	72.88%	76.51%
FedRGL	85.33%	77.02%	79.77%

**Figure 4: Accuracy Curves of FedRGL Variants on NCI1**

6 Conclusion

In this paper, we propose a novel Federated Riemannian Graph Learning framework, namely FedRGL. FedRGL addresses geometric heterogeneity by embedding each client’s graph into a personalized mixed-curvature Riemannian space, while simultaneously capturing fine-grained local topology through a novel Ricci-gated graph convolutional network. Extensive experiments across diverse federated graph learning scenarios demonstrate that FedRGL significantly outperforms state-of-the-art FGL methods on the performance of each heterogeneous client.

References

- [1] BACHMANN, G., BÉCIGNEUL, G., AND GANEA, O. Constant curvature graph convolutional networks. In *International conference on machine learning* (2020), PMLR, pp. 486–496.
- [2] BAEK, J., JEONG, W., JIN, J., YOON, J., AND HWANG, S. J. Personalized subgraph federated learning. In *International conference on machine learning* (2023), PMLR, pp. 1396–1415.
- [3] CHAMI, I., YING, Z., RÉ, C., AND LESKOVEC, J. Hyperbolic graph convolutional neural networks. *Advances in neural information processing systems* 32 (2019).
- [4] DU, H., LIU, C., LIU, H., DING, X., AND HUO, H. An efficient federated learning framework for graph learning in hyperbolic space. *Knowledge-Based Systems* 289 (2024), 111438.
- [5] FORMAN. Bochner’s method for cell complexes and combinatorial ricci curvature. *Discrete & Computational Geometry* 29, 3 (2003), 323–374.
- [6] GANEA, O., BÉCIGNEUL, G., AND HOFMANN, T. Hyperbolic neural networks. *Advances in neural information processing systems* 31 (2018).
- [7] GHOSH, A., CHUNG, J., YIN, D., AND RAMCHANDRAN, K. An efficient framework for clustered federated learning. *Advances in neural information processing systems* 33 (2020), 19586–19597.
- [8] GU, A., SALA, F., GUNEL, B., AND RÉ, C. Learning mixed-curvature representations in product spaces. In *International conference on learning representations* (2018).
- [9] HAMILTON, W. L. *Graph representation learning*. Morgan & Claypool Publishers, 2020.
- [10] HE, C., BALASUBRAMANIAN, K., CEYANI, E., YANG, C., XIE, H., SUN, L., HE, L., YANG, L., YU, P. S., RONG, Y., ET AL. Fedgraphnn: A federated learning system and benchmark for graph neural networks. *arXiv preprint arXiv:2104.07145* (2021).
- [11] JOST, J., AND LIU, S. Ollivier’s ricci curvature, local clustering and curvature-dimension inequalities on graphs. *Discrete & Computational Geometry* 51, 2 (2014), 300–322.
- [12] KAIROUZ, P., MCMAHAN, H. B., AVENT, B., BELLET, A., BENNIS, M., BHAGOJI, A. N., BONAWITZ, K., CHARLES, Z., CORMODE, G., CUMMINGS, R., ET AL. Advances and open problems in federated learning. *Foundations and trends® in machine learning* 14, 1–2 (2021), 1–210.
- [13] KAZIUS, J., MCGUIRE, R., AND BURSI, R. Derivation and validation of toxicophores for mutagenicity prediction. *Journal of medicinal chemistry* 48, 1 (2005), 312–320.
- [14] KIM, S., LEE, Y., OH, Y., LEE, N., YUN, S., LEE, J., KIM, S., YANG, C., AND PARK, C. Subgraph federated learning for local generalization. *arXiv preprint arXiv:2503.03995* (2025).
- [15] LI, A., SUN, J., ZENG, X., ZHANG, M., LI, H., AND CHEN, Y. Fedmask: Joint computation and communication-efficient personalized federated learning via heterogeneous masking. In *Proceedings of the 19th ACM conference on embedded networked sensor systems* (2021), pp. 42–55.
- [16] LI, T., SAHU, A. K., ZAHEER, M., SANJABI, M., TALWALKAR, A., AND SMITH, V. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* 2 (2020), 429–450.
- [17] LIN, Y., LU, L., AND YAU, S.-T. Ricci curvature of graphs. *Tohoku Mathematical Journal, Second Series* 63, 4 (2011), 605–627.
- [18] LIU, R., XING, P., DENG, Z., LI, A., GUAN, C., AND YU, H. Federated graph neural networks: Overview, techniques, and challenges. *IEEE transactions on neural networks and learning systems* (2024).
- [19] MCMAHAN, B., MOORE, E., RAMAGE, D., HAMPSON, S., AND Y ARCAS, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics* (2017), PMLR, pp. 1273–1282.
- [20] NI, C.-C., LIN, Y.-Y., LUO, F., AND GAO, J. Community detection on networks with ricci flow. *Scientific reports* 9, 1 (2019), 9984.
- [21] NICKEL, M., AND KIELA, D. Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems* 30 (2017).
- [22] OLLIVIER, Y. Ricci curvature of markov chains on metric spaces. *Journal of Functional Analysis* 256, 3 (2009), 810–864.
- [23] SAMAL, A., SREEJITH, R., GU, J., LIU, S., SAUCAN, E., AND JOST, J. Comparative analysis of two discretizations of ricci curvature for complex networks. *Scientific reports* 8, 1 (2018), 8650.
- [24] SEN, P., NAMATA, G., BILGIC, M., GETOOR, L., GALLIGHER, B., AND ELIASI-RAD, T. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.
- [25] SIA, J., JONCKHEERE, E., AND BOGDAN, P. Ollivier-ricci curvature-based method to community detection in complex networks. *Scientific reports* 9, 1 (2019), 9800.
- [26] SUN, L., HUANG, Z., WU, H., YE, J., PENG, H., YU, Z., AND YU, P. S. Deepprici: Self-supervised graph structure-feature co-refinement for alleviating over-squashing. In *2023 IEEE International Conference on Data Mining (ICDM)* (2023), IEEE, pp. 558–567.
- [27] SUN, L., WANG, F., YE, J., PENG, H., AND PHILIP, S. Y. Congregate: Contrastive graph clustering in curvature spaces. In *IJCAI* (2023), pp. 2296–2305.
- [28] SUN, L., ZHANG, Z., YE, J., PENG, H., ZHANG, J., SU, S., AND YU, P. S. A self-supervised mixed-curvature graph neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2022), vol. 36, pp. 4146–4155.
- [29] SUN, L., ZHANG, Z., ZHANG, J., WANG, F., PENG, H., SU, S., AND YU, P. S. Hyperbolic variational graph neural network for modeling dynamic graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 35, pp. 4375–4383.
- [30] TAN, Y., LIU, Y., LONG, G., JIANG, J., LU, Q., AND ZHANG, C. Federated learning on non-iid graphs via structural knowledge sharing. In *Proceedings of the AAAI conference on artificial intelligence* (2023), vol. 37, pp. 9953–9961.
- [31] TOPPING, J., DI GIOVANNI, F., CHAMBERLAIN, B. P., DONG, X., AND BRONSTEIN, M. M. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522* (2021).
- [32] WALE, N., WATSON, I. A., AND KARYPIS, G. Comparison of descriptor spaces for chemical compound retrieval and classification. *Knowledge and Information Systems* 14, 3 (2008), 347–375.
- [33] WEBER, M., JOST, J., AND SAUCAN, E. Forman-ricci flow for change detection in large dynamic data sets. *Axioms* 5, 4 (2016), 26.
- [34] WU, C., WU, F., CAO, Y., HUANG, Y., AND XIE, X. Fedgnn: Federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925* (2021).
- [35] WU, C., WU, F., LYU, L., HUANG, Y., AND XIE, X. Communication-efficient federated learning via knowledge distillation. *Nature communications* 13, 1 (2022), 2032.
- [36] WU, C., WU, F., LYU, L., QI, T., HUANG, Y., AND XIE, X. A federated graph neural network framework for privacy-preserving personalization. *Nature Communications* 13, 1 (2022), 3091.
- [37] WU, Z., PAN, S., CHEN, F., LONG, G., ZHANG, C., AND YU, P. S. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [38] XIE, H., MA, J., XIONG, L., AND YANG, C. Federated graph classification over non-iid graphs. *Advances in neural information processing systems* 34 (2021), 18839–18852.
- [39] YANG, M., ZHOU, M., PAN, L., AND KING, I. khgcn: Tree-likeness modeling via continuous and discrete curvature learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (2023), pp. 2965–2977.
- [40] YAO, Y., JIN, W., RAVI, S., AND JOE-WONG, C. Fedgcn: Convergence-communication tradeoffs in federated training of graph convolutional networks. *Advances in neural information processing systems* 36 (2023), 79748–79760.
- [41] YE, Z., LIU, K. S., MA, T., GAO, J., AND CHEN, C. Curvature graph network. In *International conference on learning representations* (2019).
- [42] ZHANG, J., HUA, Y., WANG, H., SONG, T., XUE, Z., MA, R., AND GUAN, H. Fedala: Adaptive local aggregation for personalized federated learning. In *Proceedings of the AAAI conference on artificial intelligence* (2023), vol. 37, pp. 11237–11244.
- [43] ZHANG, K., YANG, C., LI, X., SUN, L., AND YU, S. M. Subgraph federated learning with missing neighbor generation. *Advances in neural information processing systems* 34 (2021), 6671–6682.
- [44] ZHONG, Z., BAO, W., WANG, J., ZHU, X., AND ZHANG, X. Flee: A hierarchical federated learning framework for distributed deep neural network over cloud, edge, and end device. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 5 (2022), 1–24.
- [45] ZHU, J., ROSSI, R. A., RAO, A., MAI, T., LIPKA, N., AHMED, N. K., AND KOUTRA, D. Graph neural networks with heterophily. In *Proceedings of the AAAI conference on artificial intelligence* (2021), vol. 35, pp. 11168–11176.
- [46] ZHU, J., YAN, Y., ZHAO, L., HEIMANN, M., AKOGLU, L., AND KOUTRA, D. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.
- [47] ZHU, W., XU, Y., HUANG, X., MIN, Q., AND ZHOU, X. Spherical graph embedding for item retrieval in recommendation system. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (2022), pp. 4752–4756.

Table 5: Importation Notations.

Symbol	Description
Federated Learning Framework	
C	Total number of clients.
$c \in \{1, \dots, C\}$	Index for a client.
$\mathcal{D}_c = (G_c, Y_c)$	Local graph dataset of client c .
θ_s	Shared GNN parameters aggregated at the server.
θ_c	Local copy of shared GNN parameters on client c .
w_c	Aggregation weight for client c .
R	Total number of communication rounds.
E	Number of local training epochs.
Personalized Riemannian Geometry	
$\mathcal{M}_c$	Personalized product manifold for client c .
$\mathcal{H}_{K_c^h}, \mathcal{S}_{K_c^s}, \mathbb{R}^{d_e}$	Hyperbolic, spherical, and Euclidean component manifolds.
d_h, d_s, d_e	Dimensions of the component manifolds (shared).
$\mathcal{K}_c = \{K_c^h, K_c^s\}$	Personalized curvature parameters for client c (local).
$\otimes$	Cartesian product of manifolds.
$\oplus, $	Vector concatenation.
$\exp_{o,K}(\cdot)$	Exponential map at the origin in a manifold with curvature parameter K .
$\log_{o,K}(\cdot)$	Logarithmic map at the origin in a manifold with curvature parameter K .
Graph and GNN Components	
$G_c = (V_c, E_c)$	Local graph data for client c .
$\mathbf{x}_u \in \mathbb{R}^d$	Raw Euclidean feature vector for node u .
$\mathbf{h}_u \in \mathcal{M}_c$	Mixed-curvature representation of node u .
$\mathbf{h}_u^e, \mathbf{h}_u^h, \mathbf{h}_u^s$	Euclidean, hyperbolic, and spherical components of $\mathbf{h}_u$.
$\mathbf{h}_u^{(l)}$	Representation of node u at layer l .
$\mathbf{m}_v^{(l+1)}$	Message from node v at layer $l + 1$.
$Ric_c(u, v)$	Discrete Ricci curvature of edge (u, v) on client c .
$g_{uv}^{(l+1)}$	Ricci gate value for edge (u, v) at layer $l + 1$.
$\alpha_{uv}^{(l+1)}$	Final aggregation weight for neighbor v on node u .
$\bigoplus_K, \otimes_K$	Manifold-native addition and scalar multiplication for curvature K .
$\mathcal{L}_{\text{task}}$	Downstream task loss.
$\mathcal{L}_{GC}$	Geometric contrastive learning loss.
$\mathbf{z}_u \in \mathcal{T}_o \mathcal{M}$	Node representation projected to the tangent space at the origin.
$\text{sim}(\cdot, \cdot)$	Similarity function in the tangent space.
τ	Temperature hyperparameter for contrastive loss.

A Notation

Our notation table is in Table 5

B Manifold-Specific Operations

In this part, we provide the detailed mathematical formalisms for the key operations used in our FedRGL framework, which are performed independently within each component manifold of the product space $\mathcal{M}_c$.

B.1 Summary of Operations in Component Manifolds

In this work, we adopt the κ -stereographic model to define Riemannian algebraic operations across both positively and negatively

curved spaces within a unified framework. This model eliminates the need for separate mathematical formulations for different geometries. The κ -stereographic model provides a unified framework for both hyperbolic $\kappa < 0$ and spherical $\kappa > 0$ geometries. The operations in this model smoothly degenerate to their Euclidean counterparts as the curvature $\kappa = 0$. Table 6 summarizes the key operations used in our framework.

B.2 Aggregation Formulas

The aggregation operation, which represents a generalized weighted average on a manifold, is central to our Ricci-Gated GCN. The algebraic implementations for each component manifold are as follows:

Table 6: Summary of Operations in Component Manifolds. Operations for hyperbolic and spherical spaces are unified in the κ -stereographic model.

Operation	Formalism in Euclidean Space $\mathbb{R}^d$	Unified Formalism in κ -stereographic model ($\mathcal{H}_\kappa^d / \mathcal{S}_\kappa^d$)
Distance Metric	$d_\kappa(\mathbf{x}, \mathbf{y}) = \ \mathbf{x} - \mathbf{y}\ _2$	$d_\kappa(\mathbf{x}, \mathbf{y}) = \frac{2}{\sqrt{ \kappa }} \tanh_\kappa^{-1}(\sqrt{ \kappa } \ \mathbf{x} \oplus_\kappa \mathbf{y}\ _2)$
Exponential Mapping	$\exp_\kappa^\kappa(\mathbf{v}) = \mathbf{x} + \mathbf{v}$	$\exp_\kappa^\kappa(\mathbf{v}) = \mathbf{x} \oplus_\kappa \left(\tanh_\kappa \left(\sqrt{ \kappa } \frac{\lambda_\kappa^\kappa \ \mathbf{v}\ _2}{2} \right) \frac{\mathbf{v}}{\sqrt{ \kappa } \ \mathbf{v}\ _2} \right)$
Logarithmic Mapping	$\log_\kappa^\kappa(\mathbf{y}) = \mathbf{x} - \mathbf{y}$	$\log_\kappa^\kappa(\mathbf{y}) = \frac{2}{\lambda_\kappa^\kappa \sqrt{ \kappa }} \tanh_\kappa^{-1}(\sqrt{ \kappa } \ \mathbf{x} \oplus_\kappa \mathbf{y}\ _2) \frac{-\mathbf{x} \oplus_\kappa \mathbf{y}}{\ \mathbf{x} \oplus_\kappa \mathbf{y}\ _2}$
Addition ($\oplus_\kappa$)	$\mathbf{x} \oplus_\kappa \mathbf{y} = \mathbf{x} + \mathbf{y}$	$\mathbf{x} \oplus_\kappa \mathbf{y} = \frac{(1+2\kappa\langle\mathbf{x}, \mathbf{y}\rangle + \kappa\ \mathbf{y}\ ^2)\mathbf{x} + (1-\kappa\ \mathbf{x}\ ^2)\mathbf{y}}{1+2\kappa\langle\mathbf{x}, \mathbf{y}\rangle + \kappa^2\ \mathbf{x}\ ^2\ \mathbf{y}\ ^2}$
Scalar Multiplication ($\otimes_\kappa$)	$r \otimes_\kappa \mathbf{x} = r\mathbf{x}$	$r \otimes_\kappa \mathbf{x} = \tanh_\kappa(r \cdot \tanh_\kappa^{-1}(\ \mathbf{x}\ _2)) \frac{\mathbf{x}}{\ \mathbf{x}\ _2}$

- Aggregation in Euclidean Space $\mathbb{R}^{de}$: The aggregation is the standard weighted vector sum.

$$\mathbf{a}_u^e = \sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{uv} \mathbf{m}_v^e. \quad (17)$$

- Aggregation in Hyperbolic Space $\mathcal{H}_{K_c}^{dh}$: Aggregation is computed as the Einstein midpoint. A numerically stable approach is to first map points from the Lorentz model to the Klein model, perform a weighted average, and then map the result back to the Lorentz model. Given messages $\mathbf{m}_v^h = [m_{v,0}^h, m_{v,1:d_h}^h] \in \mathcal{H}_{K_c}^{dh}$, the aggregation is computed in two steps:

(1) **Compute the aggregated vector.** $\tilde{\mathbf{a}}_u^K$ in the Klein model's coordinates:

$$\tilde{\mathbf{a}}_u^K = \sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{uv} \left(\sqrt{K_c^h} \frac{\mathbf{m}_{v,1:d_h}^h}{m_{v,0}^h} \right). \quad (18)$$

(2) **Map the result back to the Lorentz model to obtain the final aggregated representation:**

$$\mathbf{a}_u^h = \left[\sqrt{K_c^h + \|\tilde{\mathbf{a}}_u^K\|_2^2}, \tilde{\mathbf{a}}_u^K \right]. \quad (19)$$

- Aggregation in Spherical Space $\mathcal{S}_{K_c}^{ds}$: Aggregation is computed by taking the weighted sum of vectors in the ambient space $\mathbb{R}^{d_s+1}$ and then projecting the result back onto the sphere.

$$\mathbf{a}_u^s = \sqrt{K_c^s} \frac{\sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{uv} \mathbf{m}_v^s}{\|\sum_{v \in \mathcal{N}(u) \cup \{u\}} \alpha_{uv} \mathbf{m}_v^s\|_2}. \quad (20)$$

C Additional Experimental Results

C.1 Computational and Communication Costs Analysis

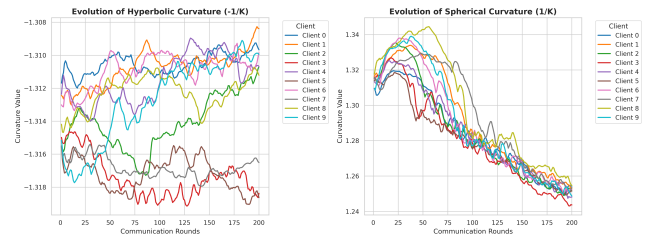
To analyze the computational and communication costs, we compare FedRGL with the baselines used in the main text on the Cora dataset, measuring the resources required to reach 80% accuracy, as summarized in Table 7.

FedRGL introduces a mixed-curvature embedding space and a Ricci-gated convolution module, which increase model capacity and per-round computational overhead. This results in slightly higher

per-round training time (1.99s) and peak memory consumption (152.43MB) compared to lightweight baselines such as FedAvg (0.5s, 76.78MB) and GCFL (0.44s, 84.97MB).

The key strengths of FedRGL lie in its significantly reduced communication rounds and overall communication overhead. As shown in Table 7, FedRGL achieves the target accuracy in only 4 communication rounds, far fewer than other methods (10–59 rounds). Although its per-round runtime is higher, the total training time (7.94s) remains competitive, and its total communication volume is the lowest (49.76MB)—less than 11% of FedHGCN and 27% of FedSage+. This efficiency stems from the strong representational capacity of the mixed-curvature space and the topology-aware Ricci gating, which together promote rapid convergence. Additionally, personalized curvature parameters are updated locally and never transmitted to the server, effectively controlling per-round communication load.

In conclusion, while FedRGL introduces a manageable increase in per-round complexity, its remarkable convergence speed makes it an efficient and practical framework.

**Figure 5: Personalized and Adaptive Curvature Learning Across Federated Clients(10 clients)**

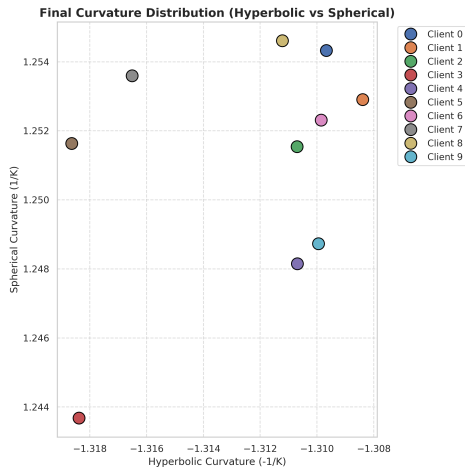
C.2 Client-side curvature learning

We conducted a client-side curvature learning experiment where ten clients with non-IID data partitions independently and continuously learned their own curvature parameters for hyperbolic and spherical spaces over 200 communication rounds.

The evolution plots demonstrate this personalization. While all ten clients begin training with similar curvature values, their respective spherical and hyperbolic curvature trajectories diverge

Table 7: Computational and Communication Costs Comparison (80% Accuracy)

Method	Total Rounds	Time/Round (s)	Total Time (s)	Max Memory (MB)	Comm./Round (MB)	Total Comm. (MB)
FedAvg	10	0.5	5.01	76.78	7.67	76.71
FedProx	14	0.62	8.67	84.09	7.67	107.40
FedSage+	12	0.43	5.20	173.50	15.29	183.52
GCFL	11	0.44	4.79	84.97	7.67	84.38
FedHGCN	59	1.25	73.55	208.36	7.81	460.95
FedLoG	17	0.42	7.19	95.57	7.67	130.41
FedRGL	4	1.99	7.94	152.43	12.44	49.76

**Figure 6: Dispersed Final Curvature Distribution Across Federated Clients(10 clients)**

significantly throughout the optimization process, ultimately converging to distinct, stable points. This indicates that each client, based on its unique local data structure and geometry, has learned a tailored curvature. The final scatter plot of curvature distribution provides definitive evidence, as the ten client points are clearly separated across the 2D plane, with no two clients converging to the same pair of curvature values, confirming successful independent personalization.

In conclusion, our method effectively addresses data heterogeneity in federated learning by enabling genuine personalised curvature learning. It avoids the sub-optimal constraint of a single, global curvature, allowing each client to converge to a curvature pair that best fits its local graph structure.